

自动驾驶汽车的仿真设计与应用

Gazebo是一个3D动力学仿真器，能够准确的高效的模拟工作在复杂室内和室外环境的机器人。最早是在2002年，由南加州大学的Andrew Howard和他的学生Nate Koenig开发的。在接下来的几年中Nate在完成他的PhD学位的同时开发着Gazebo。在2009年，Willow公司的一个高级研究工程师JohnHsu，把ROS和PR2与Gazebo整合在一起。自此Gazebo就成为了ROS社区中的一个基本工具。在2011年的春天，Willow Garage公司开始为Gazebo的开发提供商业支持。在2012年，开源机器人联盟(Open Source Robotics Foundation, OSRF)从Willow Garage公司脱离出来，同时接管了Gazebo项目。

Gazebo功能：

- 构建机器人运动仿真模型：
在Gazebo里，提供了最基础的三个物体，球体，圆柱体，立方体，利用这三个物体以及它们的伸缩变换或者旋转变换，可以设计一个最简单的机器人三维仿真模型。更加地，Gazebo提供了CAD，Blender等各种2D，3D设计软件的接口，可以导入这些图纸让Gazebo的机器人模型更加真实，这些之后会详细介绍。同时，Gazebo提供了机器人的运动仿真，通过Model Editor下的plugin，来添加我们需要验证的算法文件，就可以在Gazebo里对机器人的运动进行仿真
- 构建现实世界各种场景的仿真模型：
Gazebo可以建立一个用来测试机器人的仿真场景，通过添加物体库，放入垃圾箱，雪糕桶，甚至是人偶等物体来模拟现实世界，还可以通过Building Editor，添加2D的房屋设计图，在设计图基础上构建出3D的房屋
- 构建传感器仿真模型：
Gazebo拥有一个很强大的传感器模型库，包括camera，depth camera，laser，imu等机器人常用的传感器，并且已经有模拟库，已经可以直接使用，也可以自己从0创建一个新的传感器，添加它的具体参数，甚至还可以添加传感器噪声模型，让传感器更加真实
- 为机器人模型添加现实世界的物理性质：
Gazebo里有force，physics的选项，可以为机器人添加例如重力，阻力等，Gazebo有一个很接近真实的物理仿真引擎，要记得一般的地面是没有阻力的，和现实世界有区别

本次任务要求同学们使用自动驾驶汽车中的gazebo仿真，来进行简单汽车模型的设计和运行。

二、任务目标

- 了解gazebo仿真的功能，界面，和特点。
- 能够用gazebo仿真创建简单模型。

三、任务环境

- 操作系统：Ubuntu20.04 ROS-neotic
- 工具软件：NoMachine
- 必备硬件：自动驾驶汽车

四、任务原理

4.1 gazebo简介

4.1.1 认识gazebo

Gazebo是一款3D动态模拟器，能够在复杂的室内和室外环境中准确有效地模拟机器人群。与游戏引擎提供高保真度的视觉模拟类似，Gazebo提供高保真度的物理模拟，其提供一整套传感器模型，以及对用户和程序非常友好的交互方式。

基本应用：

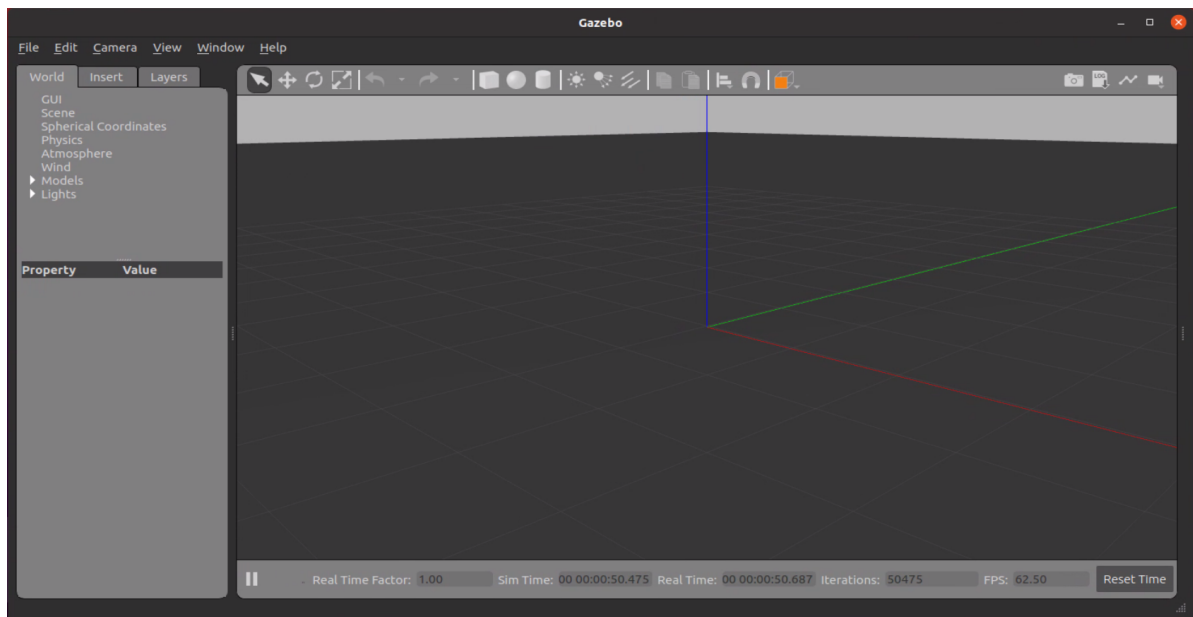
- 测试机器人算法
- 设计机器人
- 用显示场景进行回归测试

基本特点：

- 包含多个物理引擎
- 包含丰富的机器人模型和环境库
- 包含多样的传感器
- 程序设计便捷，图形界面UI合理

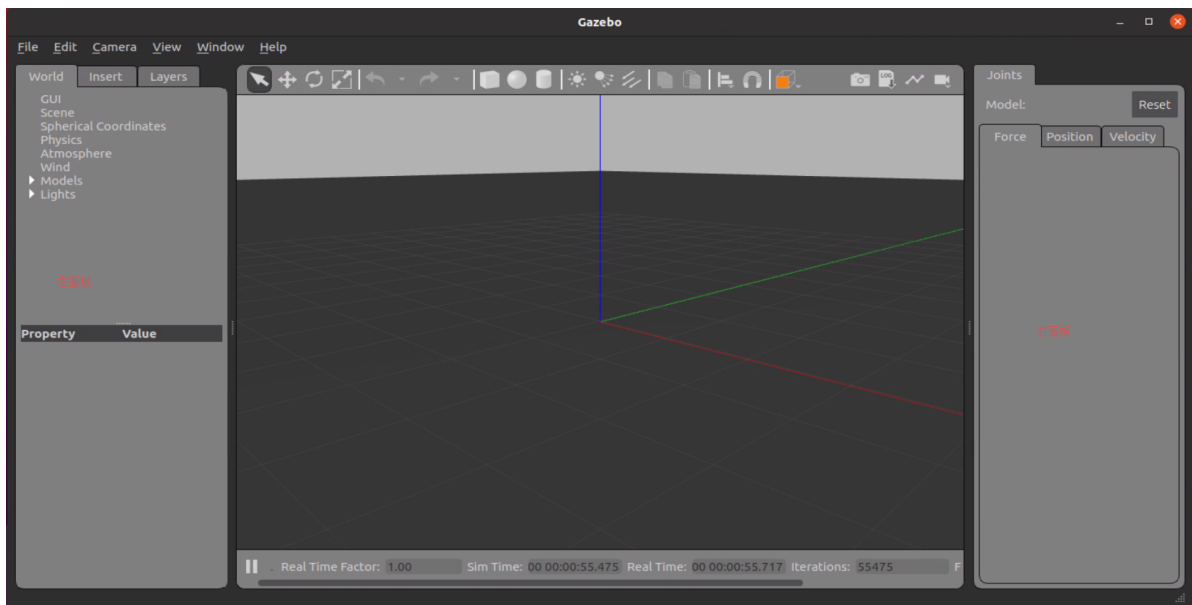
4.1.2 GUI界面详解

打开终端，输入 gazebo 即可；当然也可以通过图形化标签搜索找到并固定到任务栏，基本场景界面如下：



场景是模拟器的主要部分，用于显示仿真模型，操作仿真对象与环境进行交互。

面板



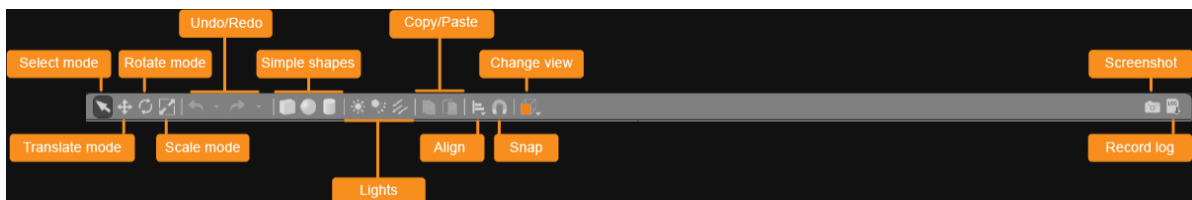
左面板：

- WORLD：“世界”选项卡，显示当前在场景中的模型，并允许你查看和修改模型参数，例如它们的姿势。你还可以通过展开“GUI”选项并调整相机姿势来更改摄像机视角。
- INSERT：“插入”选项卡，向模拟添加新对象（模型）。要查看模型列表，您可能需要单击箭头以展开文件夹。在要插入的模型上单击（和释放），然后在场景中再次单击以添加它。
- LAYERS：“图层”选项卡可组织和显示模拟中可用的不同可视化组（如果有）。图层可以包含一个或多个模型。打开或关闭图层将显示或隐藏该图层中的模型。大多数情况下都是空的，现在可以无视掉

右面板：

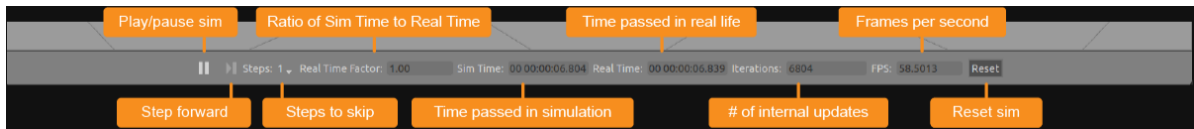
- 默认情况下Gazebo界面隐藏右侧面板。单击并拖动栏以将其打开。右侧面板可用于与所选模型（joint）的移动部件进行交互。如果未在场景中选择任何模型，则面板不会显示任何信息。

上下工具栏(Toolbars)与菜单栏(menu)



顶部工具栏：

- 选择模型（Select mode）：在场景中做标注
- 转换模式（Translate mode）：选择要移动模型，可以沿x轴，y轴，z轴，或任意方向移动模型
- 旋转模式（Rotate mode）：选择要旋转的模型，可以沿x轴，y轴，z轴进行旋转模型
- 缩放模式（Scale mode）：选择要缩放的模型，可以沿x轴，y轴，z轴进行缩小或放大模型
- 撤消/重做（Undo/Redo）：撤消/重做场景中的操作
- 简单物体（Simple shapes）：放置一个长方体/球体/圆柱体
- 灯光（Lights）：将灯光添加到场景，点光源（球状点光源）/聚光灯（从上而下，金字塔状向下照射）/方向性光源（平行光）
- 复制/粘贴（Copy/Paste）：复制/粘贴场景中的模型
- 对齐（Align）：将模型彼此对齐
- 捕捉（Snap）：将一个模型捕捉到另一个模型
- 更改视图（Change view）：从不同角度查看场景



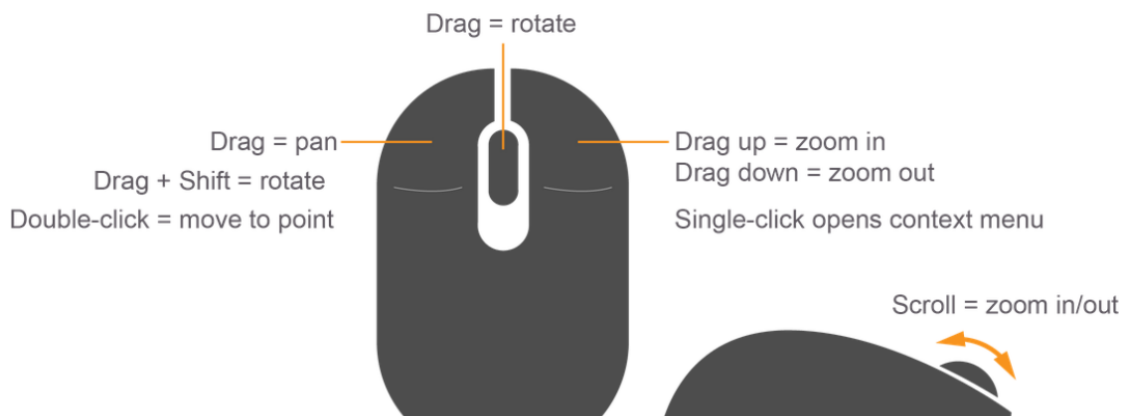
底部工具栏：

- 播放/暂停：根据你的程序，运行仿真或暂停
- 以步长播放：按照规定的时长，一步一步运行仿真，默认步长1ms
- 实时因子：模拟时间与真实时间的比率，1代表实时模拟
- 仿真时间（Simulation time）：当仿真运行时，时间在仿真环境中过得有多快。仿真可以比真实时间慢或快，具体取决于运行仿真所需的计算量
- 真实时间（Real time）：在仿真环境中运行时实际经过的时间。仿真时间和真实时间的比率称为实时因子
- 迭代次数：世界状态每迭代一次就更新一次。您可以在底部工具栏的右侧看到迭代次数。每次迭代都会将仿真推进一个固定的秒数，称为步长

菜单栏：

像大多数应用程序一样，gazebo顶部有一个应用程序菜单。某些菜单选项会显示工具栏中。在场景中，右键单击上下文菜单选项，可查看各种菜单。注意：有些Linux桌面会隐藏应用程序菜单。如果看不到菜单，请将光标移到应用程序窗口的顶部，菜单就会出现。

鼠标：



具体使用方法上图已有详述，包含在场景中导航和更改视角的基本鼠标操作。右键单击模型将打开一个包含各种选项的上下文菜单。

4.2 urdf与sdf

4.2.1 urdf是什么

统一机器人描述格式（urdf）是ROS用于描述机器人的所有元素的XML文件格式。要在gazebo中使用urdf文件，必须添加一些特定用于仿真的标签才能与gazebo一起正常使用。

尽管urdf在ROS中是一种有用且标准化的格式，但它们缺少许多功能，并且尚未进行更新以应对机器人技术的不断发展的需求。urdf只能单独指定单个机器人的运动学和动力学特性，无法指定世界中机器人本身的姿势。它也不是通用的描述格式，因为它不能指定关节环（平行连接），并且缺乏摩擦和其他特性。此外，它不能指定非机器人，例如灯光，高度图等。

在实现方面，urdf语法大量使用XML属性破坏了正确的格式设置，这反过来又使urdf更加不灵活。

4.2.2 sdf是什么

为了解决此问题，创建了一种称为仿真描述格式（sdf）的新格式，供gazebo使用，以解决urdf的缺点。sdf是从世界级到机器人级的所有内容的完整描述，能够描述机器人、静态和动态物体、照明、地形甚至物理学的各方面的信息。sdf可以精确描述机器人的各类性质，除了传统的运动学特性之外，还可以为机器人定义传感器、表面属性、纹理、关节摩擦等；sdf还提供了定义各种环境的方法。包括环境光照、地形等。

sdf也使用XML格式进行描述。

总结而言，sdf是urdf的进化版，能够更好的描述真实的模拟条件。

4.2.3 使用哪种格式

尽管目前有一些sdf与urdf的之间的转换方法，但往往十分复杂且易出错。因此，建议刚开始就根据自己的需求选择最合适的模型格式。

- 必须使用urdf的情况：要使用rviz进行可视化操作。
- 必须使用sdf的情况：研究并联机器人，或机器人中存在封闭链结构。
- 建议使用urdf的情况：要尽快做出仿真用以演示效果；使用Solidworks建模，想方便地导出ROS三维模型。
- 建议使用sdf的情况：想深入研究ROS-gazebo仿真，使仿真的动力学特性更加真实；想开发自己专用的Gazebo仿真插件。

4.2.4 urdf转sdf

由于sdf是gazebo的原生格式，因此urdf转sdf是比较简单的。采用的方法一般是：使用常规方法将.urdf加载到gazebo中后，再将其另存为一个单独的.world文件。此时sdf格式的模型就完整地保存在*.world文件的元素下了。

另外，gazebo官方还提供了另一种命令行方法，也可将实现urdf转sdf。在urdf所在目录下打开终端，执行如下命令：

```
$ gz sdf -p my_model.urdf > my_model.sdf
```

4.2.5 sdf转urdf

虽然官方没有给出将sdf转为urdf的方法，但好在有大佬自己开发了可行的工具——pysdf功能包，github链接：<https://link.zhihu.com/?target=https%3A/github.com/andreasBihlmaier/pysdf>

该包的版本过老，在ubuntu18.04和ros melodic上直接运行报错，对其做了一些细微的修改后可用：<https://link.zhihu.com/?target=https%3A/github.com/chenjm1109/pysdf>

该功能包使用起来也很简单，分为4个步骤：

- 在github中下载pysdf功能包，放到ROS工作空间（以catkin_ws为例）下的src目录下；
- 编译工作空间

- ```
$ cd ~/catkin_ws/
$ catkin_make
```

- 从待转换的sdf模型文件所在的目录打开终端，执行如下命令

- ```
$ rosrun pysdf sdf2urdf.py my_model.sdf result_model.urdf
```

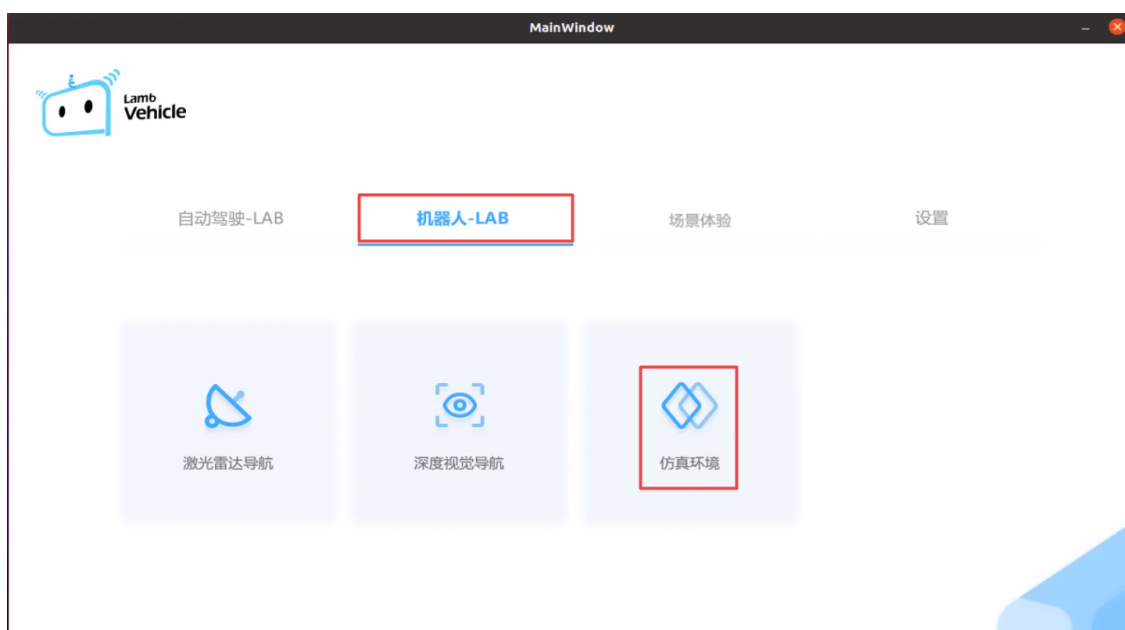
注意事项：

- sdf2urdf.py是python可执行文件，如果报错[roslaunch] Couldn't find executable named sdf2urdf below..., 就需要先通过 `chmod +x *` 指令为其赋予可执行权限，这是使用所有ROS-python可执行文件时都要注意的事情。
 - sdf文件中不要有插件，也不要ball关节等urdf无法识别的关节类型。
 - 转换完成后可使用 `check_urdf` 工具检查urdf的合法性，命令如下：
- ```
$ sudo apt-get install liburdfdom-tools
$ check_urdf result_model.rudf
```

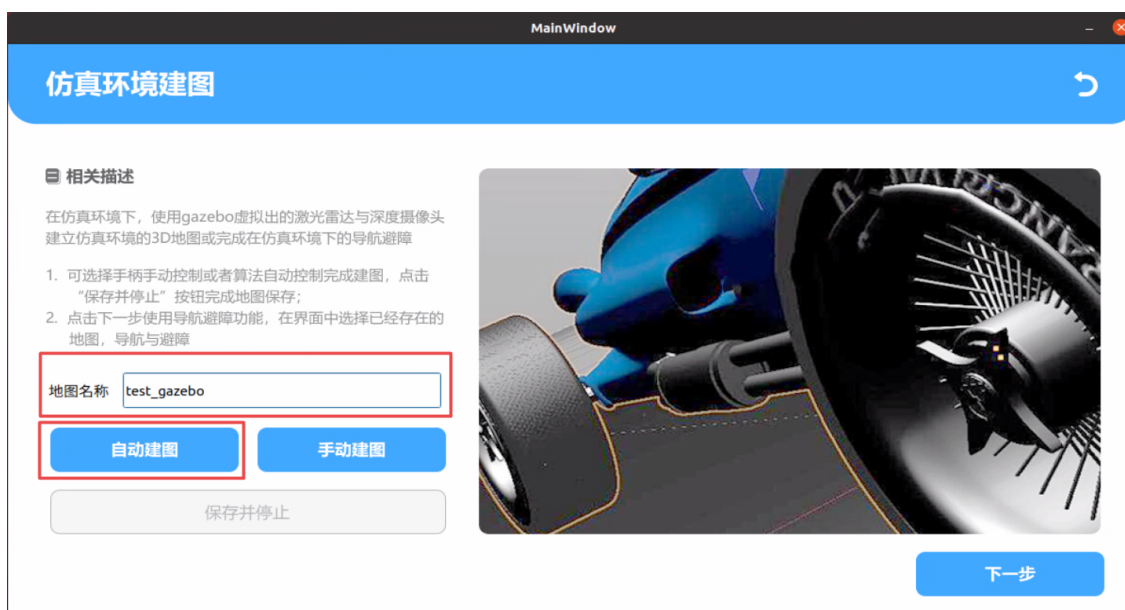
## 五、任务步骤

### 步骤一 gazebo自动建图

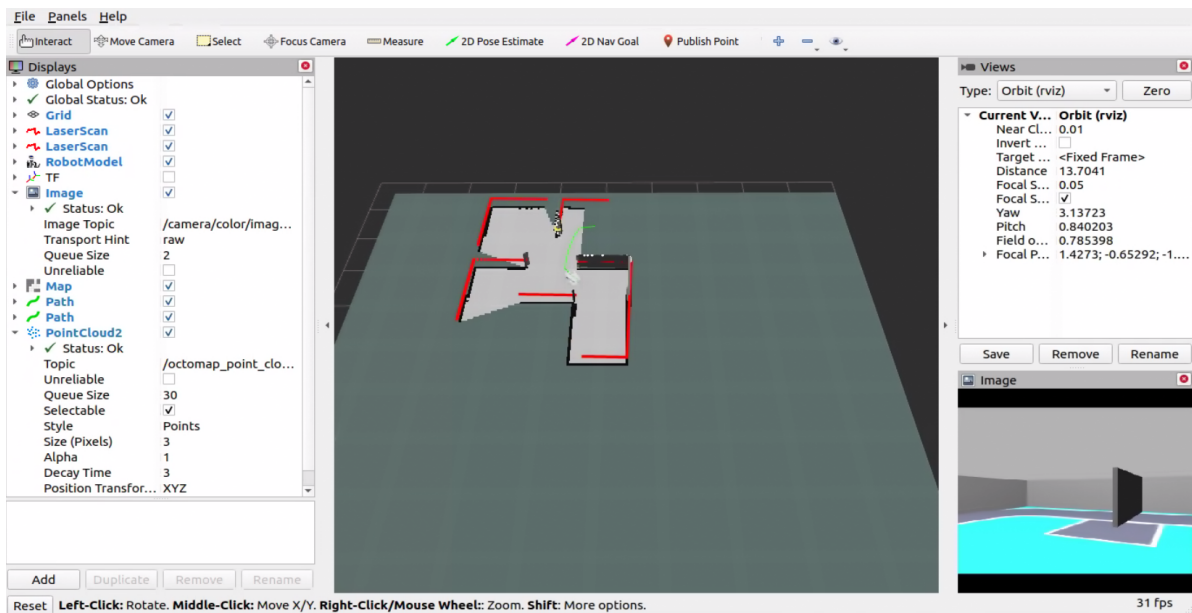
1. 打开ros2.0小车桌面应用，选择“机器人-LAB”→“仿真环境”



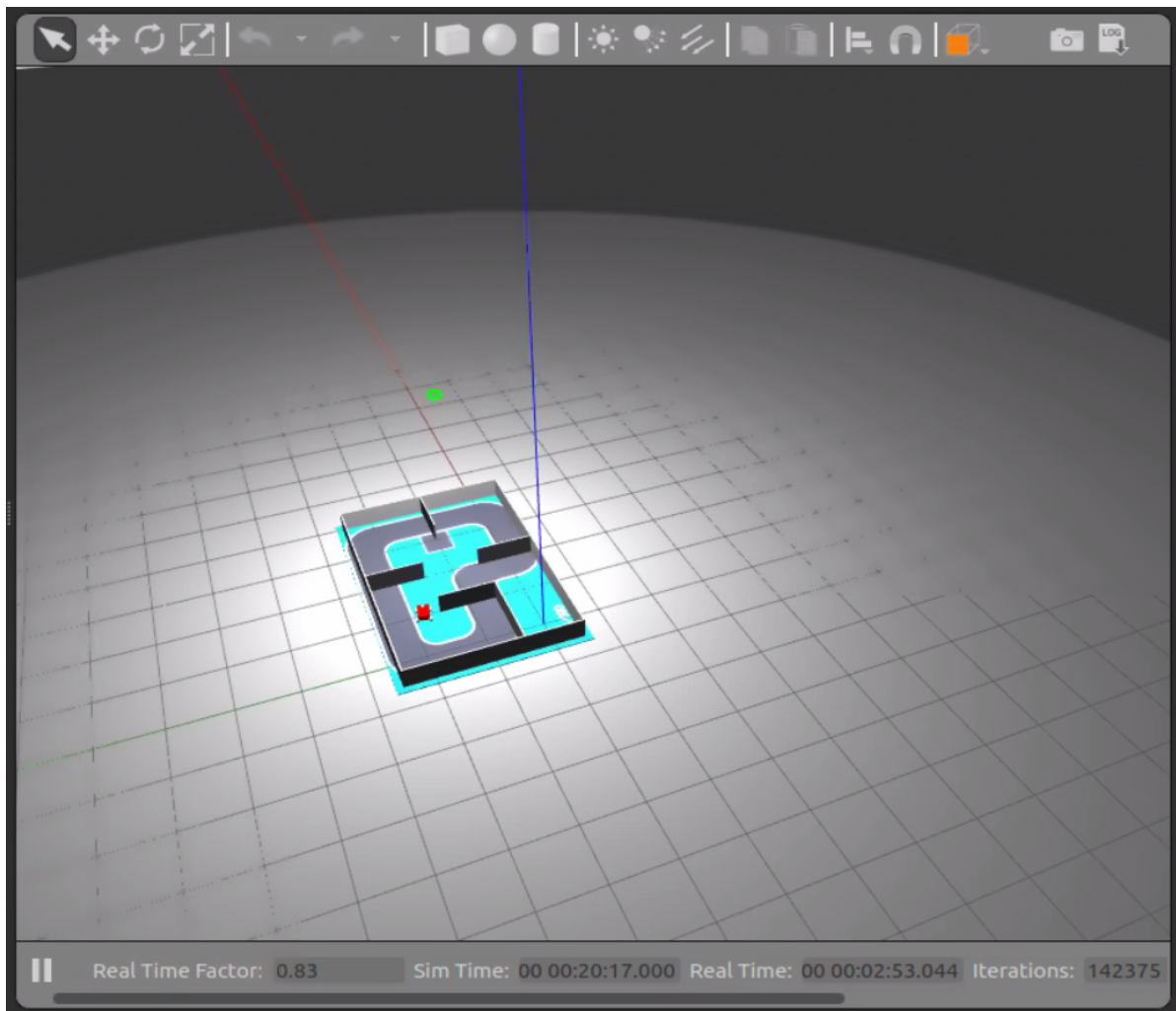
2. 进入到仿真环境建图页面，输入地图名字，点击“自动建图”开始建图，手动建图请参考真实地图手动建图部分。



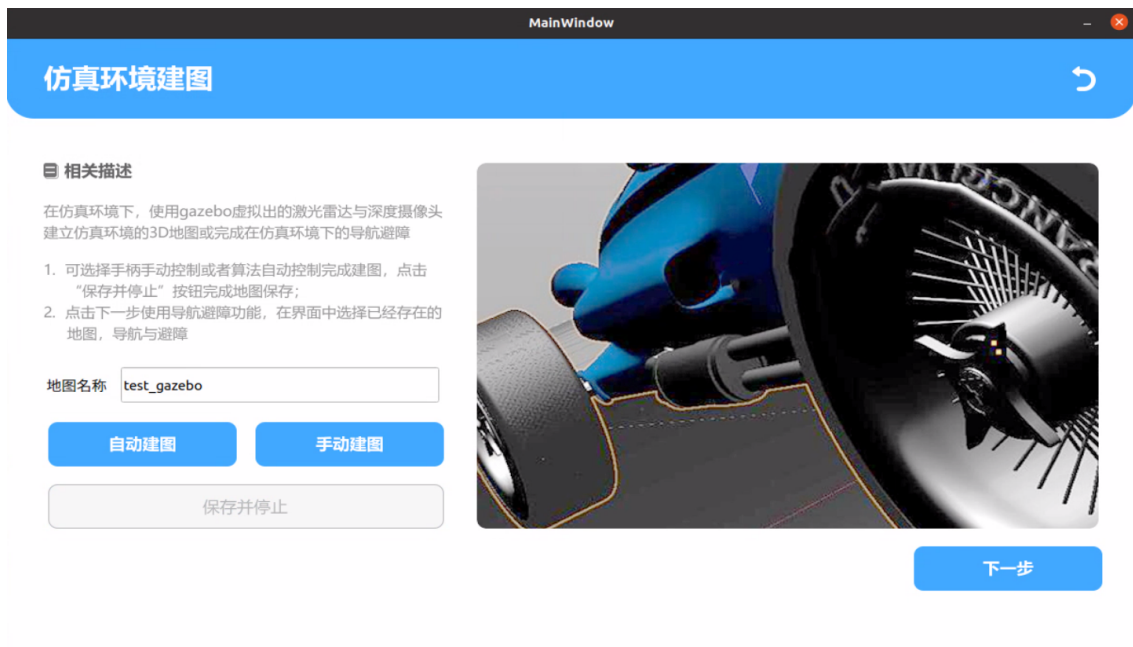
在打开的RVIZ，可以看到小车在地图中沿着地图的四个角进行自动行进建图，右下角为摄像头视角：



同时，可以看到打开的gazebo中，小车同步行进：



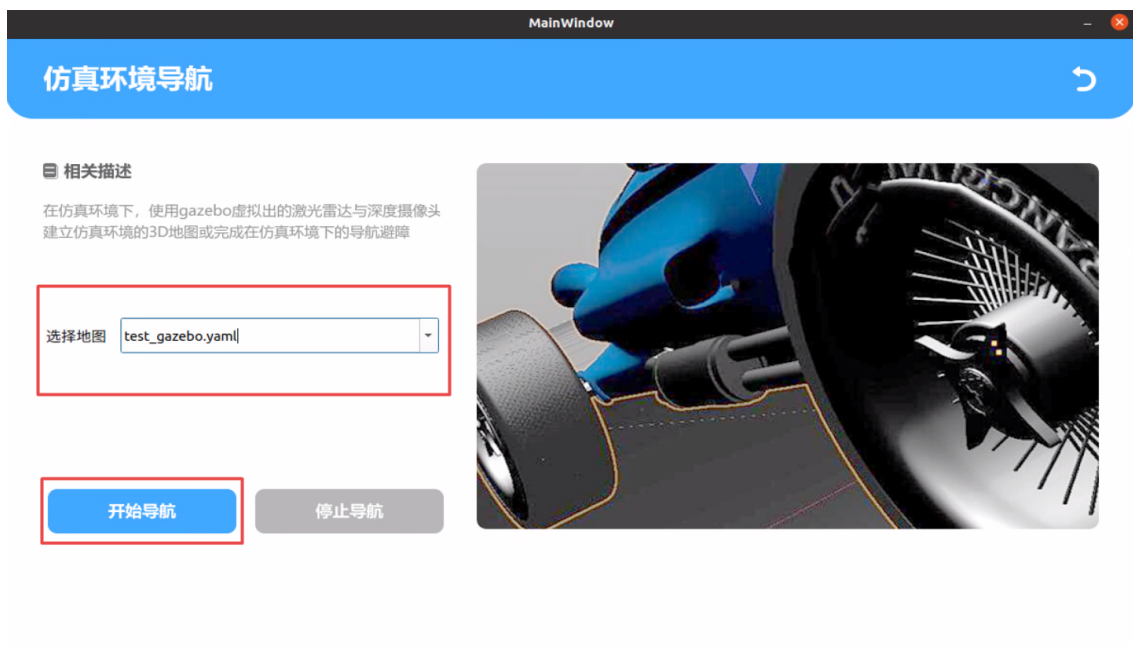
3. 建图完成，回到小车2.0桌面应用，点击“保存并停止”，保存地图。



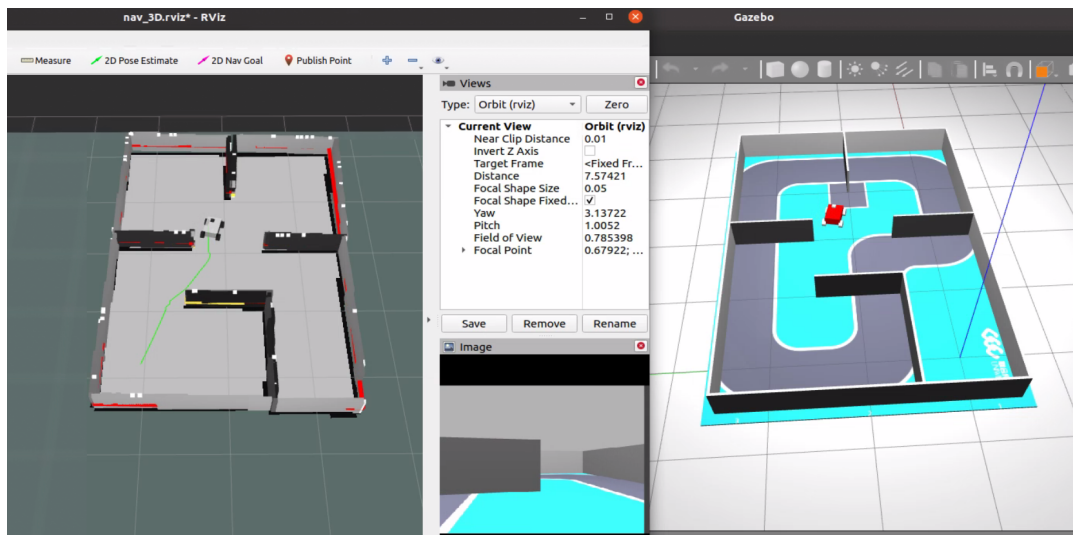
接下来可以使用创建好的地图进行自动导航。

## 步骤二 gazebo导航

1. 在保存地图界面点击“下一步”，进入自动导航页面。



2. 选择创建好的地图，点击“开始导航”，进入到导航功能。



自动导航会打开RVIZ和gazebo，可以看到小车在仿真地图中同步行进。

## 步骤三 代码实现

### 1. 创建建图启动节

点"/home/mo/ros\_space/src/ai\_node\_master/launch/mapping\_gaz\_explore.launch"，添加如下内容：

```
<!-- 2D建图节点 -->
<include file="$(find
navigation_settings)launch/robot_gmapping.launch.xml"/>

<!-- gazebo仿真节点 -->
<include file="$(find urdf_car)launch/unionws_gazebo.launch"/>

<!-- 3D建图节点 -->
<!-- <include file="$(find rtabmap_local)/launch/rgbd_mapping.launch" /> -->
<include file="$(find octomap_local)/launch/octomap_mapping.launch" />

<node pkg="ai_node_master" type="3d_map_saver.py" name="map_saver_3d"
output="screen">
 <param name="map_name" value="$(arg map)"/>
</node>

<!-- 避障节点 -->
<include file="$(find navigation_settings)launch/move_base.launch.xml" >
 <arg name="3d_enable" default="$(arg 3d_enable)" />
```

```

 <arg name="mapping" default="$(arg mapping)" />
</include>

<!-- 自动导航节点 -->
<!-- <include file="$(find rrt_exploration)/launch/simple.launch" > -->
<include file="$(find loop_nav)/launch/loop_nav_3d.launch" >
 <arg name="rotate_speed" default="$(arg rotate_speed)" />
 <arg name="rotation_flage" default="$(arg rotation_flage)" />
 <arg name="nav_sim_flag" default="$(arg nav_sim_flag)" />
</include>

<!-- 深度图像转化雷达数据 -->
<include file="$(find ai_node_master)launch/depth_to_scan.launch">
 <arg name="scan_height" default="50" />
</include>

<!-- 可视化界面 -->
<node pkg="rviz" type="rviz" name="rviz" args="-d $(find
urdf_car)/rviz/mapping_3D.rviz" output="screen"/>

```

其中

是启动gazebo节点，其余节点请参照之前任务。

2. 创建节点"/home/mo/ros\_space/src/urdf\_car/launch/unionws\_gazebo.launch"，添加如下内容：

```

<!-- 包含gazebo空地图节点 -->
<include file="$(find gazebo_ros)/launch/empty_world.launch">
 <!-- 引入全局地图 -->
 <arg name="world_name" value="$(find urdf_car)/worlds/map.world"/>
 <arg name="debug" value="$(arg debug)" />
 <arg name="gui" value="$(arg gui)" />
 <arg name="paused" value="$(arg paused)"/>
 <arg name="use_sim_time" value="$(arg use_sim_time)"/>
 <arg name="headless" value="$(arg headless)"/>
</include>

```

```

<!-- 通过car_control.yaml发布所有节点状态 -->
<rosparam file="$(find urdf_car)/unionws_control/config/car_control.yaml"
command="load"/>

<!-- <arg name="model" default="$(find robot_sim_demo)/urdf/robot.xacro"/> -
->
<!-- 引入小车模型 -->
<arg name="model" default="$(find urdf_car)/urdf/unionws.xacro"/>

```

```

<!-- 调用ROS的spawner节点 -->
<node pkg="controller_manager" ns="/unionws" type="spawner"
name="joint_state_controller_spawner" respawn="false" output="screen"
 args="joint_state_controller" />

<param name="robot_description" command="$(find xacro)/xacro $(arg model)"
/>
<param name="use_gui" value="false"/>

<!-- 该节点通过读取参数服务器上的/robot_description来获得机器人的所有非固定关节，并
从/move_group/fake_controller_joint_states主题来获取关 节状态。主要功能是负
责输出关节消息（sensor_msgs/JointState），并发布/joint_state话题到/move_group
和/robot_state_publisher节点。 -->
<node name="joint_state_publisher" pkg="joint_state_publisher"
type="joint_state_publisher" />
<!-- 接收joint_state_publisher发布的话题消息，并通过tf发布结果到/move_group节点 --
>
<node name="robot_state_publisher" pkg="robot_state_publisher"
type="robot_state_publisher" />

<!-- 添加小车模型 -->
<node name="urdf_spawner" pkg="gazebo_ros" type="spawn_model"
respawn="false" output="screen"
 args="-urdf -model mybot -param robot_description -x 0 -y 0 -z 0"/>

```

3. 创建"/home/mo/ros\_space/src/urdf\_car/unionws\_control/config/car\_control.yaml"文件，添加如下内容，用来发布说有节点状态信息：

```

unionws:
 # Publish all joint states -----
 joint_state_controller:
 type: joint_state_controller/JointStateController
 publish_rate: 50

```

4. 创建小车模型文件"/home/mo/ros\_space/src/urdf\_car/urdf/unionws.xacro"，添加如下内容，用来描述小车的具体外观：

```

<xacro:include filename="$(find urdf_car)/urdf/unionws.gazebo" />
<xacro:include filename="$(find urdf_car)/urdf/materials.xacro" />

```

```

<link name = "base_link">
 <visual>
 <geometry>
 <box size="0.3 .2 .1"/>
 </geometry>
 <origin rpy = "0 0 0" xyz = "0 0 0.05"/>
 <material name="white">
 <color rgba="1 1 1 1"/>
 </material>
 </visual>
 <!-- <collision>
 <geometry>
 <box size="0.3 .2 .1"/>
 </geometry>
 <origin rpy = "0 0 0" xyz = "0 0 0.05"/>
 </collision> -->

```

```

 <inertial>
 <origin rpy="0 0 0" xyz="0 0 0"/>
 <mass value="0.00003542" />
 <inertia ixx="0.0045" ixy="0" ixz="0" iyy="0.005" iyz="0"
izz="0.0045" />
 </inertial>

</link>

<link name = "f_left_wheel">

 <visual>
 <geometry>
 <cylinder length="0.05" radius="0.05"/>
 </geometry>
 <origin rpy = "1.5707 0 0" xyz = "0 0 0"/>
 <material name="black">
 <color rgba="0 0 0 1"/>
 </material>
 </visual>
 <collision>
 <geometry>
 <cylinder length="0.05" radius="0.05"/>
 </geometry>
 <origin rpy = "1.5707 0 0" xyz = "0 0 0"/>
 </collision>
 <inertial>
 <mass value="0.00003542" />
 <origin rpy="0 0 0" xyz="0 0 0"/>
 <inertia ixx="1" ixy="0" ixz="0" iyy="1" iyz="0" izz="1" />
 </inertial>
</link>

<link name = "f_right_wheel">
 <visual>
 <geometry>
 <cylinder length="0.05" radius="0.05"/>
 </geometry>
 <origin rpy = "1.5707 0 0" xyz = "0 0 0"/>
 <material name="black">
 <color rgba="0 0 1 1"/>
 </material>
 </visual>
 <collision>
 <geometry>
 <cylinder length="0.05" radius="0.05"/>
 </geometry>
 <origin rpy = "1.5707 0 0" xyz = "0 0 0"/>
 </collision>
 <inertial>
 <mass value="0.00003542" />
 <origin rpy="0 0 0" xyz="0 0 0"/>
 <inertia ixx="1" ixy="0" ixz="0" iyy="1" iyz="0" izz="1" />
 </inertial>
</link>

<link name = "b_left_wheel">

```

```

<visual>
 <geometry>
 <cylinder length="0.05" radius="0.05"/>
 </geometry>
 <origin rpy = "1.5707 0 0" xyz = "0 0 0"/>
 <material name="black">
 <color rgba="0 1 0 1"/>
 </material>
</visual>
<collision>
 <geometry>
 <cylinder length="0.05" radius="0.05"/>
 </geometry>
 <origin rpy = "1.5707 0 0" xyz = "0 0 0"/>
</collision>
<inertial>
 <mass value="0.00003542" />
 <origin rpy="0 0 0" xyz="0 0 0"/>
 <inertia ixx="1" ixy="0" ixz="0" iyy="1" iyz="0" izz="1" />
</inertial>
</link>

<link name = "b_right_wheel">
 <visual>
 <geometry>
 <cylinder length="0.05" radius="0.05"/>
 </geometry>
 <origin rpy = "1.5707 0 0" xyz = "0 0 0"/>
 <material name="black">
 <color rgba="1 0 0 1"/>
 </material>
 </visual>
 <collision>
 <geometry>
 <cylinder length="0.05" radius="0.05"/>
 </geometry>
 <origin rpy = "1.5707 0 0" xyz = "0 0 0"/>
 </collision>
 <inertial>
 <mass value="0.00003542" />
 <origin rpy="0 0 0" xyz="0 0 0"/>
 <inertia ixx="1" ixy="0" ixz="0" iyy="1" iyz="0" izz="1" />
 </inertial>
</link>

```

```

<link name="base_footprint"> </link>
<joint name="base_footprint_joint" type="fixed">
 <parent link="base_footprint"/>
 <child link="base_link"/>
</joint>

```

```

<joint name = "f_left_motor" type= "continuous">

```

```

 <origin rpy = "0 0 0" xyz = "0.125 0.11 0"/>
 <parent link = "base_link"/>
 <child link = "f_left_wheel"/>
 <axis xyz="0 1 0"/>
</joint>

<joint name = "f_right_motor" type= "continuous">
 <origin rpy = "0 0 0" xyz = "0.125 -0.11 0"/>
 <parent link = "base_link"/>
 <child link = "f_right_wheel"/>
 <axis xyz="0 1 0"/>
</joint>

<joint name = "b_left_motor" type= "continuous">
 <origin rpy = "0 0 0" xyz = "-0.125 0.11 0"/>
 <parent link = "base_link"/>
 <child link = "b_left_wheel"/>
 <axis xyz="0 1 0"/>
</joint>

<joint name = "b_right_motor" type= "continuous">
 <origin rpy = "0 0 0" xyz = "-0.125 -0.11 0"/>
 <parent link = "base_link"/>
 <child link = "b_right_wheel"/>
 <axis xyz="0 1 0"/>
</joint>

```

```

<!-- IMU -->
<link name="imu_link"/>
<joint name="base_to_imu" type="fixed">
 <axis rpy="0 0 0" xyz="0 0 0"/>
 <parent link="base_link"/>
 <child link="imu_link"/>
 <origin rpy="0 0 0" xyz="0 0 .1"/>
</joint>

<!-- camera -->
<link name="camera_link"/>
<joint name="base_to_cam" type="fixed">
 <axis rpy="0 0 0" xyz="0 0 0"/>
 <parent link="base_link"/>
 <origin rpy="0 0 0" xyz="0.16 0 0.2"/>
 <child link="camera_link"/>

</joint>

<link name="camera_link_fake"></link>
<joint name="camera_joint_fake" type="fixed">
 <origin xyz="0 0 0" rpy="-1.5708 0 -1.5708 "/>
 <parent link="camera_link"/>
 <child link="camera_link_fake"/>
</joint>

```

```

<!-- radar -->

```

```

<link name = "radar">
 <visual>
 <geometry>
 <box size="0.05 .05 .02"/>
 </geometry>
 <origin rpy = "0 0 0" xyz = "0 0 0"/>
 <material name="black">
 <color rgba="0 0 0 1"/>
 </material>
 </visual>

 <collision>
 <geometry>
 <box size="0.05 .05 .02"/>
 </geometry>
 <origin rpy = "0 0 0" xyz = "0 0 0"/>
 </collision>
</link>
<joint name = "radar_joint" type= "fixed">
 <origin rpy = "0 0 3.1415" xyz = "0 0 0.2"/>
 <parent link = "base_link"/>
 <child link = "radar"/>
</joint>

<transmission name="b_left_trans">
 <type>transmission_interface/SimpleTransmission</type>
 <joint name="b_left_motor">

<hardwareInterface>hardware_interface/EffortJointInterface</hardwareInterface>
e>
 </joint>
 <actuator name="b_leftMotor">

<hardwareInterface>hardware_interface/EffortJointInterface</hardwareInterface>
e>
 <mechanicalReduction>1</mechanicalReduction>
 </actuator>
</transmission>

<transmission name="f_left_trans">
 <type>transmission_interface/SimpleTransmission</type>
 <joint name="f_left_motor">

<hardwareInterface>hardware_interface/EffortJointInterface</hardwareInterface>
e>
 </joint>
 <actuator name="f_leftMotor">

<hardwareInterface>hardware_interface/EffortJointInterface</hardwareInterface>
e>
 <mechanicalReduction>1</mechanicalReduction>
 </actuator>
</transmission>

<transmission name="b_right_trans">
 <type>transmission_interface/SimpleTransmission</type>
 <joint name="b_right_motor">

```

```

<hardwareInterface>hardware_interface/EffortJointInterface</hardwareInterface>
</joint>
<actuator name="b_rightMotor">

<hardwareInterface>hardware_interface/EffortJointInterface</hardwareInterface>
</actuator>
</transmission>

<transmission name="f_right_trans">
 <type>transmission_interface/SimpleTransmission</type>
 <joint name="f_right_motor">

<hardwareInterface>hardware_interface/EffortJointInterface</hardwareInterface>
</joint>
<actuator name="f_rightMotor">

<hardwareInterface>hardware_interface/EffortJointInterface</hardwareInterface>
</actuator>
</transmission>

```

## 5. 创建自动导航启动节

点"/home/mo/ros\_space/src/ai\_node\_master/launch/navigation\_gaz.launch"，添加如下内容，实现gazebo仿真自动导航：

```

<launch>
 <!--param参数配置-->
 <param name="node_id_closer" value="/navigation_gaz" />
 <arg name="map" default="mymap2.yaml" />
 <arg name="rotate_speed" default="0.22" />
 <arg name="rotation_flg" default="false" />
 <arg name="nav_sim_flag" default="true" />
 <arg name="3d_enable" default="true" />
 <arg name="mapping" default="false" />
 <node pkg="ai_node_master" type="ros_closer" name="navigation_gaz"
output="screen" required="true" />

 <!-- gazebo仿真节点 -->
 <include file="$(find urdf_car)launch/unionws_gazebo.launch"/>

 <!-- 运行地图服务器，并且加载设置的地图-->
 <node name="map_server" pkg="map_server" type="map_server" args="$(find
ai_node_master)/maps/$(arg map)"/>

 <node pkg="ai_node_master" type="3d_map_publisher.py" name="map_publisher_3d"
output="screen">
 <param name="map_name" value="$(arg map)"/>
 </node>

 <!-- 避障节点 -->
 <include file="$(find navigation_settings)launch/move_base.launch.xml" >

```

```

 <arg name="3d_enable" default="$(arg 3d_enable)" />
 <arg name="mapping" default="$(arg mapping)" />
 </include>

 <!-- 启动AMCL节点 -->
 <arg name="custom_amcl_launch_file" default="$(find
navigation_settings)launch/robot_amcl.launch.xml"/>
 <include file="$(arg custom_amcl_launch_file)"></include>

 <!-- 自动导航节点 -->
 <include file="$(find loop_nav)/launch/loop_nav_3d.launch" >
 <arg name="rotate_speed" default="$(arg rotate_speed)" />
 <arg name="rotation_flage" default="$(arg rotation_flage)" />
 <arg name="nav_sim_flag" default="$(arg nav_sim_flag)" />
 </include>

 <!-- 手动设定多点导航节点 -->
 <!-- <node pkg="loop_nav" type="send_mark_reverse.py" name="send_mark"
output="screen">
 <param name="reverse_flag" value="true"/>
 </node> -->

 <!-- 深度图像转化雷达数据 -->
 <include file="$(find ai_node_master)launch/depth_to_scan.launch">
 <arg name="scan_height" default="50" />
 </include>

 <!-- 可视化界面 -->
 <node pkg="rviz" type="rviz" name="rviz" args="-d $(find
urdf_car)/rviz/nav_3D.rviz" output="screen"/>

</launch>

```

其中gazebo启动节点参考本任务自动导航部分，其余节点参考真实地图自动导航部分。

## 六、任务总结

通过本次任务的学习，同学们能够认识机器人3D仿真平台gazebo，并且能够创建属于自己的模型，方便自动驾驶课程的学习和实验。本次内容只涉及到gazebo的部分基础内容，同学们可以深入学习，基于gazebo仿真平台创建更加复杂的模型，调试出更优的运行算法。